

VM Execution Flow

This diagram illustrates the execution flow within the BDI Virtual Machine.

Execution Sequence

```
sequenceDiagram
    participant User
    participant VM as Virtual Machine
    participant Interp as Interpreter
    participant Hot as Hot Path Detector
    participant JIT as JIT Compiler
    participant GC as Garbage Collector
    participant Native as Native Code

    User->>VM: Execute(bytecode)
    VM->>VM: Initialize execution context

    alt Cold Code Path
        VM->>Interp: Interpret bytecode
        Interp->>Hot: Update execution count
        Hot->>Hot: Check if hot (count < threshold)
        Interp->>VM: Return result
    else Hot Code Path
        VM->>Hot: Check execution count
        Hot->>Hot: Count >= threshold (hot!)
        Hot->>JIT: Trigger compilation

        alt Baseline Tier
            JIT->>JIT: Fast compilation (1-10ms)
            JIT->>Native: Generate baseline code
        else Optimized Tier
            JIT->>JIT: Full optimization (10-100ms)
            JIT->>Native: Generate optimized code
        end

        Native->>VM: Execute native code
        VM->>VM: Return result
    end

    alt Memory Allocation Needed
        VM->>GC: Allocate object
        GC->>GC: Check if collection needed

        alt Collection Needed
            GC->>GC: Perform garbage collection
            GC->>VM: Return allocated memory
        else No Collection
            GC->>VM: Return allocated memory
        end
    end

    VM->>User: Return execution result
```

Execution States

State 1: Initialization

```
stateDiagram-v2
    [*] --> Idle
    Idle --> Loading: Load bytecode
    Loading --> Ready: Bytecode loaded
    Ready --> Executing: Start execution
```

State 2: Execution Mode Selection

```
stateDiagram-v2
    Executing --> CheckHotPath: Check execution count
    CheckHotPath --> Interpreting: Cold code
    CheckHotPath --> Compiling: Hot code

    Interpreting --> UpdateProfile: Update statistics
    UpdateProfile --> CheckHotPath: Continue

    Compiling --> BaselineJIT: Tier 1
    Compiling --> OptimizedJIT: Tier 2

    BaselineJIT --> NativeExecution: Code ready
    OptimizedJIT --> NativeExecution: Code ready

    NativeExecution --> CheckRecompile: Check performance
    CheckRecompile --> OptimizedJIT: Needs optimization
    CheckRecompile --> [*]: Complete

    Interpreting --> [*]: Complete
```

Detailed Execution Flow

1. Bytecode Interpretation

```

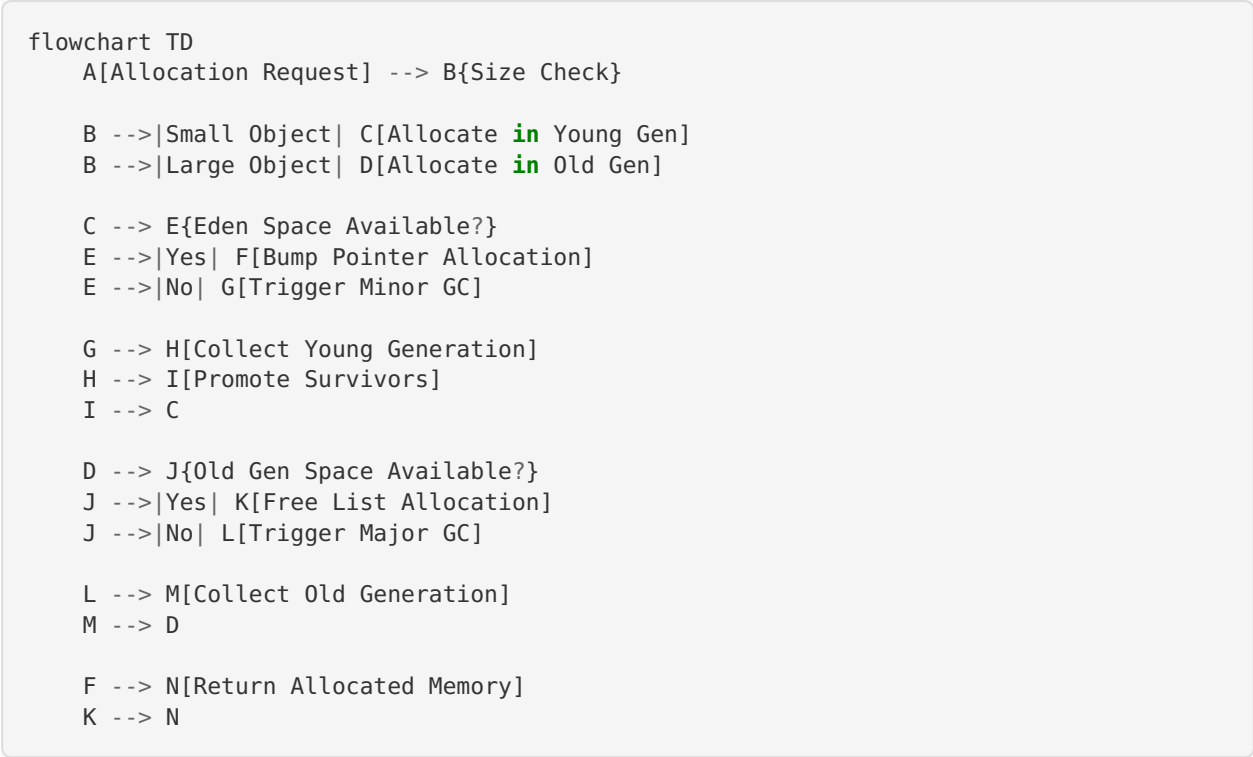
flowchart TD
    A[Start Interpretation] --> B[Fetch Instruction]
    B --> C[Decode Opcode]
    C --> D{Instruction Type}
    D -->|Arithmetic| E[Execute Arithmetic]
    D -->|Control Flow| F[Execute Branch/Jump]
    D -->|Memory| G[Execute Load/Store]
    D -->|Call| H[Execute Function Call]
    E --> I[Update Stack]
    F --> J[Update IP]
    G --> K[Access Memory]
    H --> L[Push Call Frame]
    I --> M{More Instructions?}
    J --> M
    K --> M
    L --> M
    M -->|Yes| B
    M -->|No| N[Return Result]
  
```

2. JIT Compilation Trigger

```

flowchart TD
    A[Execute Function] --> B[Increment Execution Count]
    B --> C{Count >= Baseline Threshold?}
    C -->|No| D[Continue Interpretation]
    C -->|Yes| E{Already Compiled?}
    E -->|Yes| F{Count >= Optimized Threshold?}
    E -->|No| G[Trigger Baseline Compilation]
    F -->|No| H[Execute Baseline Code]
    F -->|Yes| I[Trigger Optimized Compilation]
    G --> J[Compile to Baseline]
    I --> K[Compile to Optimized]
    J --> H
    K --> L[Execute Optimized Code]
    D --> M[Update Profile]
    H --> M
    L --> M
    M --> N[Continue Execution]
  
```

3. Memory Allocation Flow



Performance Characteristics

Execution Modes

Mode	Startup Time	Execution Speed	Use Case
Interpreter	0 ms	10-50x slower	Cold code, startup
Baseline JIT	1-10 ms	2-5x slower	Warm code
Optimized JIT	10-100 ms	0.8-1.2x native	Hot code

Transition Thresholds

Interpreter → Baseline JIT: 100 executions
Baseline JIT → Optimized JIT: 1000 executions

Stack Frame Layout

Return Address
Previous Frame Pointer
Local Variable N Local Variable N-1 ... Local Variable 1 Local Variable 0
Temporary Value M Temporary Value M-1 ... Temporary Value 1 Temporary Value 0

[Back to Architecture Overview](#) (../README.md)